1- **CLASS TASKS SOLUTIONS**

--- TASK 1, 2, 3 & 4 SOLUTION KEY (PYTHON CODE) ---

```python
import pandas as pd
import numpy as np

# Task 1: Setup, Audit & Clean
raw_q3_txns = {
    "txn_id": ["TX-101", "TX-102", "TX-103", "TX-103", "TX-104", "TX-105",
"TX-106", "TX-107", "TX-108"],
    "facility_id": ["FAC-A", "FAC-B", "FAC-C", "FAC-C", "FAC-A", "FAC-B",
"FAC-C", "FAC-A", "FAC-B"],
    "txn_date": ["2026-07-01", "2026-07-03", "2026-07-10", "2026-07-10",
"2026-08-05", "2026-08-12", "2026-09-01", "2026-09-15", "2026-09-28"],
    "product_line": [" FOOTWEAR ", "Apparel", " accessories ", "
accessories ", "Footwear", " APPAREL ", "Accessories", "Footwear",
"Apparel"],
    "gross_sales": [1200.00, 350.00, np.nan, np.nan, 850.00, 420.00,
190.00, 950.00, 310.00],
    "units_shipped": [10, 5, 2, 2, 8, 6, 3, 9, 4]
}

raw_facility_meta = {
    "facility_id": ["FAC-A", "FAC-B", "FAC-C"],
    "region_name": ["Northeast", "Southeast", "Midwest"],
    "fulfillment_cost_per_txn": [250.00, 180.00, 220.00]
}

df_txns = pd.DataFrame(raw_q3_txns)
df_facilities = pd.DataFrame(raw_facility_meta)

# Deduplicate & Clean
df_txns = df_txns.drop_duplicates().reset_index(drop=True)
df_txns["product_line"] = df_txns["product_line"].str.strip().str.title()
df_txns["txn_date"] = pd.to_datetime(df_txns["txn_date"])

median_sales = df_txns["gross_sales"].median() # 420.00
df_txns["gross_sales"] = df_txns["gross_sales"].fillna(median_sales)

# Task 2: Merge & Feature Engineering
df_full = pd.merge(df_txns, df_facilities, on="facility_id", how="left")
df_full["net_margin"] = df_full["gross_sales"] -
df_full["fulfillment_cost_per_txn"]

# Task 3: Aggregations
region_summary = df_full.groupby("region_name").agg(
    total_gross_sales=("gross_sales", "sum"),
    total_fulfillment_cost=("fulfillment_cost_per_txn", "sum"),
    total_net_margin=("net_margin", "sum"),
    order_count=("txn_id", "count")
).sort_values(by="total_net_margin", ascending=True)

print("--- REGIONAL SUMMARY ---")
print(region_summary)
```

--- TASK 4 BUSINESS MEMO SOLUTION KEY ---

"To: Victoria Vance, VP of Retail Operations

Re: Q3 Profitability & Operational Leakage Investigation

An end-to-end EDA across audited Q3 transactions ($4,690.00 gross sales across 8 orders) identifies the Midwest region (FAC-C) as the primary driver of profit leakage. The Midwest region generated $610.00 in gross sales but accrued $440.00 in fulfillment overhead, delivering a meager $170.00 total net margin. In contrast, the Northeast region generated $2,250.00 in net margin.

Recommendations:

1. Conduct a fulfillment process audit at Midwest facility FAC-C to reduce per-transaction processing costs from $220.00 down to the Southeast baseline of $180.00.
2. Reallocate promotional marketing inventory toward high-margin Footwear products in the Northeast."

2- **<u>HOMEWORK SOLUTION</u>**

```python

```
Homework Problem 1 & 2 Python Solution
import pandas as pd
import numpy as np

raw_sub_logs = {
    "sub_id": ["SUB-301", "SUB-302", "SUB-303", "SUB-303", "SUB-304", "SUB-305", "SUB-306", "SUB-307", "SUB-308"],
    "plan_code": ["PLN-1", "PLN-2", "PLN-3", "PLN-3", "PLN-1", "PLN-2", "PLN-3", "PLN-1", "PLN-2"],
    "start_date": ["2026-01-10", "2026-02-14", "2026-03-01", "2026-03-01", "2026-04-12", "2026-05-18", "2026-06-01", "2026-07-22", "2026-08-11"],
    "user_tier": [" Enterprise ", "Pro", " basic ", " basic ", "Enterprise", " PRO ", "Basic", "Enterprise", "Pro"],
    "monthly_fee": [499.00, 99.00, np.nan, np.nan, 499.00, 99.00, 29.00, np.nan, 99.00],
    "churn_status": ["Active", "Churned", "Churned", "Churned", "Active", "Active", "Churned", "Active", "Churned"]
}

raw_plan_meta = {
    "plan_code": ["PLN-1", "PLN-2", "PLN-3"],
    "plan_name": ["Enterprise Platinum", "Pro Gold", "Basic Bronze"],
    "support_cost_per_user": [150.00, 40.00, 20.00]
}

df_subs = pd.DataFrame(raw_sub_logs).drop_duplicates().reset_index(drop=True)
df_subs["user_tier"] = df_subs["user_tier"].str.strip().str.title()

```
df_subs["start_date"] = pd.to_datetime(df_subs["start_date"])

median_fee = df_subs["monthly_fee"].median() # 99.00
df_subs["monthly_fee"] = df_subs["monthly_fee"].fillna(median_fee)

df_plans = pd.DataFrame(raw_plan_meta)
df_merged = pd.merge(df_subs, df_plans, on="plan_code", how="left")
df_merged["net_user_profit"] = df_merged["monthly_fee"] -
df_merged["support_cost_per_user"]

# Aggregations
churn_summary = df_merged.groupby("churn_status").agg(
    total_revenue=("monthly_fee", "sum"),
    total_net_profit=("net_user_profit", "sum"),
    user_count=("sub_id", "count")
)

tier_churn = df_merged.groupby(["user_tier",
"churn_status"]).size().unstack(fill_value=0)

print("--- CHURN SUMMARY ---")
print(churn_summary)
print("\n--- TIER CHURN ---")
print(tier_churn)
```

- Homework Grading Criteria:
    - Correctly removes duplicate record `SUB-303` and imputes missing fees with $99.00 median.
    - Calculates total monthly revenue lost to churn ($256.00 across 4 churned users).
    - Identifies **Pro Gold / Pro Tier** as experiencing the highest churn count (2 churned users out of 3 total = 66.7% churn rate).
    - Formulates retention strategies targeting Pro tier user onboarding and product engagement.